Assignment-3

Parameter-Efficient Fine-Tuning (PEFT) with Adapters

1. Objective

The goal is to implement a Transfer Learning pipeline where a pre-trained language model is adapted to a downstream classification task using Supervised Fine-Tuning (SFT). To mitigate the cost of training and very high storage requirements, you will implement Parameter-Efficient Fine-Tuning (PEFT) by injecting "Adapters" into the transformer architecture

2. Dataset Access

To maintain efficient training, we will use the Hugging Face SST-2 (Stanford Sentiment Treebank) dataset for binary sentiment classification**.**

Link: https://huggingface.co/datasets/stanfordnlp/sst2

You must limit your training subset to a maximum of 10,000 sentences. This ensures that the training loop finishes quickly and does not exceed the session memory limits of a standard Colab T4 GPU.

3. Implementation Tasks

Task 1: Preprocessing & BERT Tokenization

- Tokenization: Load a pre-trained bert-base-uncased tokenizer. Process the raw text into Token Embeddings, Segment Embeddings, and Position Embeddings, which are the required input configurations for BERT.
- Sequence Constraint: Set a maximum sequence length of 128 tokens to strictly prevent Out-of-Memory (OOM) errors**.**

Task 2: Building the Adapter Module

- Implement a custom Adapter block in PyTorch. , an Adapter is a modular component placed inside the Transformer.
- Architecture: Your adapter must consist of a feed-forward neural network (NN). It should project the hidden state to a smaller bottleneck dimension, apply a non-linearity, and project it back to the original hidden dimension**.**

Task 3: Injecting Adapters into the Base Model

- Freezing Weights: Load the pre-trained BERT base model. You must freeze all the base parameters of the network.
- Injection: Insert your custom Adapter modules after the Self-Attention layers within the transformer blocks. Make sure *only* the Adapter weights and the final classification head are set to require gradients, so they can be tuned to improve the model's performance on this specific task

Task 4: Supervised Fine-Tuning (SFT) & Inference

- Training Loop: Train the adapter-infused model for 3 to 5 epochs using the AdamW optimizer and Cross-Entropy Loss.
- Inference: Write a simple pipeline where a user can input a raw string (e.g., "The movie was absolutely fantastic"), and the model outputs the predicted sentiment (Positive/Negative).

4. Submission Requirements

Submit a single Colab Notebook containing:

1. Code: Full implementation of the Tokenization, Adapter Module, and Training Loop.
2. Visual Proof: A plot of the Training Loss vs. Validation Loss across the epochs showing convergence.
3. Parameter Count Comparison: Print the total number of parameters in the model versus the number of *trainable* parameters.
4. Brief Analysis (Max 250 words): explain why standard full fine-tuning poses challenges (mentioning specific costs) and describe how adding a "set of modular components inside the transformer" resolves these issues.

5. Constraints & Guidelines

- Due Date: 10 April 2026 till 11:59 P.M.
- Late Penalty: 1 mark deducted per day after the due date.
- Originality: DO NOT use LLMs or plagiarize from online repositories; the institute's plagiarism policy is strictly enforced.